CW1: Regression Model for Predicting Outcome

Mohamed Alketbi k22056537

GitHub: `https://github.com/M-alk/ML_CW1`

1 Exploratory Data Analysis

The training set contains 10,000 observations with 30 predictors (3 categorical, 27 numeric) and a continuous target (`outcome`). Initial inspection revealed no missing values. The target is approximately symmetric (skewness = 0.08) with mean ≈ -5.0 and standard deviation ≈ 12.7. No extreme outliers were found in the target variable (range $[-44.9, 39.7]$).

A correlation analysis identified `depth` as the strongest linear predictor ($r = -0.41$), followed by `b3` ($r = 0.23$), `b1` ($r = 0.17$), `a1` ($r = 0.15$), and `table` ($r = 0.11$). Several features exhibited near-zero correlation with the target: `a5`, `a6`, `a7`, `a9`, `b5`, `b6`, `b7`, and `b9` (all $|r| < 0.013$). A heatmap of inter-predictor correlations further revealed strong multicollinearity among the geometric features (`x`, `y`, `z`, `carat`, `price`), though these were retained as tree-based models handle collinearity naturally.

The three categorical features (`cut`, `color`, `clarity`) represent ordinal diamond properties. These were encoded using scikit-learn's `TargetEncoder`, which applies internal cross-validation to prevent target leakage.

2 Model Selection

Five model families were compared using 5-fold cross-validation on the full training set, with preprocessing (target encoding) inside the pipeline to avoid data leakage.

Model	**CV R^2**	**Std**
ElasticNet (baseline)	0.2890	0.0184
Random Forest	0.4573	0.0212
HistGradientBoosting	0.4710	0.0212
XGBoost	0.4744	0.0197
CatBoost	0.4793	0.0181

Table 1: Model comparison (5-fold CV, all with same preprocessing and feature set).

The linear baseline (ElasticNet) achieved $R^2 \approx 0.29$, indicating substantial non-linearity in the data. Tree-based ensembles all performed significantly better.

CatBoost was selected as the final model because it consistently achieved the highest R^2 across all tuning experiments (Figure 1), benefiting from its ordered boosting procedure and strong built-in regularisation.

3 Model Training and Evaluation

HistGradientBoosting was tuned first via sequential grid search (depth = 2 best at $R^2 = 0.4731$; leaf = 20; lr = 0.01, iter = 3,000; final $R^2 = 0.4726$). XGBoost was then tuned similarly (depth = 2, $\alpha = 1$, $\lambda = 10$, subsample = 0.7; final $R^2 = 0.4752$). CatBoost achieved the highest scores, so it was tuned most thoroughly. All experiments used the same 5-fold CV split for fair comparison.

Step 1 - Tree depth. Depths 2–6 were tested. Depth 3 yielded the best $R^2 = 0.4784$; shallower and deeper trees both performed worse.

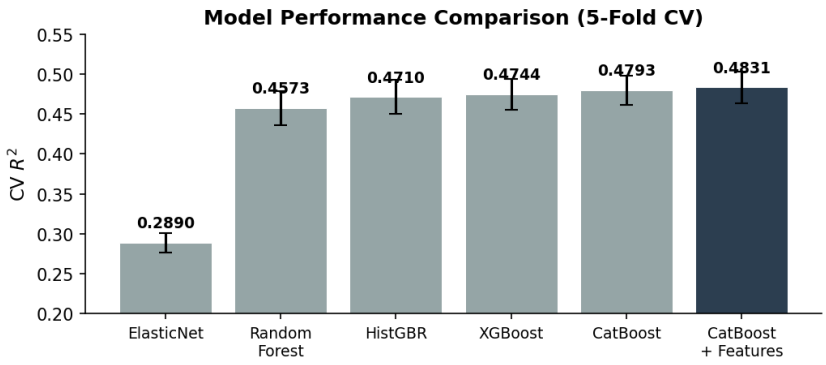

Figure 1: 5-fold CV R^2 across all models. Error bars show ±1 standard deviation.

Step 2 - Leaf regularisation. `l2_leaf_reg` values of 1, 3, 5, 10, 20, 30 were tested with depth locked at 3. A value of 30 was optimal ($R^2 = 0.4789$), indicating moderate regularisation helps.
Step 3 - Subsample ratio. Values from 0.5 to 1.0 were tested. Subsample = 0.5 was best ($R^2 = 0.4793$), introducing stochasticity that reduces overfitting.
Step 4 - Learning rate / iterations. A learning rate of 0.01 with 3,000 iterations was optimal ($R^2 = 0.4793$). Higher iteration counts (5,000) showed slight degradation.

Feature engineering. Three types of engineered features were tested: (i) polynomial/ratio features from domain knowledge (`depth`^{2}, `depth`×`table`, log(1+`price`)); (ii) interaction terms identified via correlation analysis (`b1`×`a1`, `b3`×`b1`×`a1`). An ablation study tested six candidate interactions; only `b1`×`a1` ($\Delta R^2 = -0.0040$ when removed) and `b3`×`b1`×`a1` ($\Delta R^2 = -0.0002$) were retained, while the remaining four (including `depth`/`table`) were dropped as removing them improved performance. The eight low-signal features were also dropped, which improved generalisation. Hyperparameters were re-validated on the final feature set; `l2_leaf_reg` = 30 and subsample = 0.5 remained optimal.

Final configuration: CatBoost with depth = 3, `l2_leaf_reg` = 30, subsample = 0.5, learning rate = 0.01, iterations = 3,000. With the final feature set (27 features after engineering and selection):

$$\text{CV } R^2 = 0.4831 \pm 0.0194$$

The final model was trained on all 10,000 rows and used to generate predictions on the held-out test set.

4 Code Supplement

All code is available in the project repository with the following structure:

File	Purpose
`notebooks/01_eda.ipynb`	Exploratory data analysis and visualisation
`notebooks/02_baseline.ipynb`	ElasticNet baseline model
`notebooks/03_tree_models.ipynb`	Model comparison, tuning, and feature engineering
`src/train_model.py`	Final training script (produces submission CSV)
`requirements.txt`	Python dependencies

The training script reproduces the full pipeline: data loading, feature engineering, preprocessing, cross-validation, and test set prediction. All preprocessing occurs inside the CV loop to prevent information leakage. GitHub repository: `https://github.com/M-alk/ML_CW1.git`